

Task 1 — Web App QA & Debug Report

Application Under Test: demo.realworld.io | Framework: RealWorld (Angular + Node.js)

Date: May 2025 | Tester: Srilatha Pothana | Environment: Chrome 124, Windows 11

1. Scope & Approach

The RealWorld demo (demo.realworld.io) is a Medium.com clone built to showcase real-world fullstack implementations. Testing covered all primary user flows: registration, authentication, article creation/editing/deletion, commenting, profile management, and tag-based navigation. Issues were identified via manual exploratory testing, browser DevTools (Network + Console), and edge-case input injection. No automated test tools were used. Seven issues were found across functional, security, UX, and performance categories.

2. Bug Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity
1	No client-side form validation on Sign-Up	1. Go to Sign Up page 2. Submit form with empty fields or invalid email (e.g. 'abc') 3. Observe response	Expected: Inline error messages appear immediately; form does not submit until valid Actual: Form submits to backend; error only shown after round-trip delay (1–2 s). No real-time field-level feedback	High
2	CORS error blocks all API calls intermittently	1. Open demo.realworld.io 2. Open DevTools → Console 3. Refresh page or attempt login	Expected: API calls succeed; page loads data normally Actual: Console shows: 'CORS header Access-Control-Allow-Origin missing' — entire feed fails to load	Critical
3	Logged-out users can navigate to /editor route	1. Ensure you are NOT logged in 2. Manually type /editor in the URL bar 3. Observe page behaviour	Expected: User is redirected to login page with a return URL Actual: Editor page partially renders; submitting the form triggers a silent failure with no error message	High
4	Article tags accept whitespace-only and duplicate values	1. Log in 2. Go to New Article 3. Enter ' ' (spaces) as a tag, press Enter 4. Also enter the same tag twice	Expected: Whitespace-only tags rejected; duplicates deduplicated or prevented Actual: Both whitespace and duplicate tags are accepted, stored, and displayed. Tags appear as blank chips	Medium
5	Session not cleared after logout — back-button restores authenticated state	1. Log in as a user 2. Click Log Out 3. Press browser Back button	Expected: Page shows logged-out state; protected actions (like follow, favorite) are disabled Actual: Browser cache restores previous page; user appears logged in visually; API calls then fail with 401	High
6	No pagination limit — extremely long article lists degrade	1. Open the home feed 2. Observe network tab as items load 3. Click	Expected: Consistent sub-500ms load per page; loading indicator	Medium

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity
	performance	through pages rapidly	shown Actual: No loading skeleton shown during fetch; rapid pagination triggers race conditions where older responses overwrite newer ones	
7	Password field has no minimum length enforcement	1. Go to Sign Up 2. Enter a 1-character password (e.g. 'a') 3. Complete registration	Expected: Validation error: 'Password must be at least 8 characters' Actual: Account is created successfully with a 1-character password — poses significant security risk	Critical

3. Root-Cause Analysis — Issue #2: CORS Error Blocks API Calls

Issue Summary

Users intermittently see a completely blank feed or receive an error when the app loads. Browser DevTools reveals the root cause: a Cross-Origin Resource Sharing (CORS) failure on requests to `api.realworld.io`.

What is happening?

The Angular frontend (served from `demo.realworld.io`) makes XMLHttpRequest calls to `api.realworld.io/api/*`. The browser's same-origin policy requires the backend to explicitly permit this cross-origin traffic by including the Access-Control-Allow-Origin: * (or the specific origin) response header. When this header is absent, the browser blocks the response entirely — even if the server returned a 200 OK — and the app receives no data.

Why is it happening?

The backend is hosted on a Heroku free-tier dyno. Free dynos spin down after 30 minutes of inactivity and restart on the next request. During cold-start or under memory pressure (the project maintainer confirmed abnormal memory usage in issue #1538), the Node.js server may crash or restart mid-request. When the process restarts, early responses may be sent before the CORS middleware is fully initialised, or the process may crash before writing any headers at all, causing a 400/500 with no CORS headers — which the browser treats as a CORS violation.

Contributing factors:

- No reverse proxy (e.g. Nginx) in front of the app to enforce CORS headers at the infrastructure level, independent of app state
- No health-check or keep-alive mechanism to prevent dyno sleeping
- CORS middleware is applied at the application layer, making it vulnerable to app crashes

How to fix it?

- Short-term: Move CORS header injection to the infrastructure level (e.g. an Nginx proxy or a Heroku-compatible edge layer like Cloudflare) so headers are always present regardless of backend health
- Medium-term: Upgrade the Heroku plan to a paid dyno to eliminate cold-start and memory-limit issues
- Long-term: Add a proper health-check endpoint (`/health`) and configure uptime monitoring (e.g. the n8n workflow in Task 2) to alert immediately when the backend goes down, reducing mean time to recovery

- Testing: Add a CORS smoke test to CI/CD that makes a preflight OPTIONS request and asserts the Access-Control-Allow-Origin header is present

4. Severity Summary

Severity	Count
Critical	2
High	3
Medium	2
Low	0
Total	7

Recommendation: Issues #2 (CORS) and #7 (no password minimum) are Critical and should block any production deployment. Issues #1, #3, and #5 are High priority and should be resolved before the next release cycle.